

src/

Implementation source hierarchy

A compiler-style pipeline over the ASL workflow IR — written in Rust

4 front ends · 8 pass families

COMPILER PIPELINE · data flows left → right

1 · PARSE

Front ends

src/asl.rs · dsl.rs
cncf.rs · cfn.rs

Raw ASL, typed DSL, CNCF, and CloudFormation / SAM / CDK front ends; asl.rs re-emits IR to ASL JSON.

2 · REPRESENT

Core IR

src/ir.rs

Workflow graph, annotation slots, and ASL I/O fields — the representation every pass consumes.

3 · ANALYZE

Analysis passes

src/passes/

Structural, data-flow, contract, typestate, retry, compensation, concurrency, and temporal analyses.

4 · REPORT

Diagnostics

src/diag.rs

Diagnostic records with provenance-graded severity, rendered for humans or as JSON.

DRIVER

src/main.rs

clap-driven CLI with CI-policy switches and scan / eval subcommands — orchestrates the pipeline above.

Evaluation harness · code-disjoint

src/concrete.rs

oracle

Bounded execution oracle, built disjointly from the checker to cross-validate its findings.

src/mutate.rs · annot.rs

tooling

Defect injection for the mutation study; sidecar annotation parsing feeds the semantic passes.